

COMPUTER VISION WITH OPENCV LAB EXPERIMENTS

Aim, Detailed Algorithm and Expected Output

Experiment 1A: Image to Grayscale

Aim: To read an image using Python and convert it into a grayscale image.

Algorithm:

1. Import OpenCV.
2. Read the input image using `cv2.imread()`.
3. Convert the BGR image to grayscale using `cv2.cvtColor()`.
4. Display or save the grayscale image.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: A single-channel grayscale image is obtained.

Experiment 1B: Gaussian Blur

Aim: To read an image and apply Gaussian Blur for smoothing and noise reduction.

Algorithm:

1. Read the input image.
2. Choose a Gaussian kernel size.
3. Apply `cv2.GaussianBlur()`.
4. Display the blurred image.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: A smoother image with reduced noise is obtained.

Experiment 1C: Canny Outline Detection

Aim: To detect the outline of an image using the Canny edge detection method.

Algorithm:

1. Read the image and convert it to grayscale.
2. Apply Gaussian smoothing if required.
3. Apply cv2.Canny() with suitable thresholds.
4. Display the edge image.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Major outlines and boundaries are detected.

Experiment 1D: Dilation

Aim: To dilate an image using the OpenCV dilate function.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply cv2.dilate().
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Foreground regions are expanded and nearby structures may connect.

Experiment 1E: Erosion

Aim: To erode an image using the OpenCV erode function.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply `cv2.erode()`.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Foreground regions are reduced and small bright details are removed.

Experiment 2: Basic Video Processing

Aim: To read a captured video and display it in normal, slow-motion and fast-motion modes.

Algorithm:

1. Open the video using `cv2.VideoCapture()`.
2. Read frames sequentially.
3. Change display delay to simulate slow or fast motion.
4. Display frames until the video ends.
5. Release resources.
6. Initialize the required video source and check whether it has opened successfully.
7. Read each frame continuously and verify that the frame is valid before processing.
8. Resize or normalize the frame when required for consistent processing.

Output: The same video is displayed at different playback speeds.

Experiment 3: Webcam Video Processing

Aim: To capture live video from a webcam and display it at different speeds.

Algorithm:

1. Initialize the webcam using `cv2.VideoCapture(0)`.
2. Capture frames continuously.
3. Adjust frame display delay.
4. Display the live frames.
5. Release the webcam.
6. Initialize the required video source and check whether it has opened successfully.
7. Read each frame continuously and verify that the frame is valid before processing.
8. Resize or normalize the frame when required for consistent processing.

Output: Live webcam video is displayed.

Experiment 4: Image Scaling

Aim: To resize an image to bigger and smaller dimensions.

Algorithm:

1. Read the input image.
2. Specify scaling factors or target dimensions.
3. Use `cv2.resize()` for enlargement and reduction.
4. Display the results.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Enlarged and reduced versions of the image are obtained.

Experiment 5: Image Rotation

Aim: To rotate an image in clockwise and counter-clockwise directions.

Algorithm:

1. Read the image.
2. Find the image center.
3. Create a rotation matrix using `cv2.getRotationMatrix2D()`.
4. Apply `cv2.warpAffine()`.
5. Display both rotations.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: Clockwise and counter-clockwise rotated images are obtained.

Experiment 6: Image Translation

Aim: To move an image from one position to another using translation.

Algorithm:

1. Read the image.
2. Create a translation matrix with x and y shifts.
3. Apply `cv2.warpAffine()`.
4. Display the translated image.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: The image is shifted to a new position.

Experiment 7: Affine Transformation

Aim: To perform affine transformation on an image.

Algorithm:

1. Select three source points.
2. Select three destination points.
3. Compute the affine transformation matrix.
4. Apply `cv2.warpAffine()`.
5. Display the transformed image.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: A geometrically transformed image is obtained.

Experiment 8: Perspective Transformation on Image

Aim: To perform perspective transformation on an image.

Algorithm:

1. Select four source points.
2. Select four destination points.
3. Compute the matrix using `cv2.getPerspectiveTransform()`.
4. Apply `cv2.warpPerspective()`.
5. Display the result.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: A perspective-corrected image is obtained.

Experiment 9: Perspective Transformation on Video

Aim: To perform perspective transformation on video frames.

Algorithm:

1. Open the video.
2. Read frames one by one.
3. Define source and destination points.
4. Compute the transformation matrix.
5. Apply `cv2.warpPerspective()` to each frame.
6. Display the video.
7. Initialize the required video source and check whether it has opened successfully.
8. Read each frame continuously and verify that the frame is valid before processing.

Output: A perspective-transformed video is obtained.

Experiment 10: Homography Transformation

Aim: To transform an image using a homography matrix.

Algorithm:

1. Read related images or matching points.
2. Detect or specify corresponding points.
3. Compute homography using `cv2.findHomography()`.
4. Apply `cv2.warpPerspective()`.
5. Display the aligned result.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: One image view is aligned with another.

Experiment 11: Direct Linear Transformation

Aim: To estimate a projective transformation using Direct Linear Transformation.

Algorithm:

1. Collect at least four corresponding point pairs.
2. Form the DLT equations.
3. Solve for transformation parameters.
4. Construct the homography matrix.
5. Transform the image.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: A projectively transformed image is obtained.

Experiment 12: Canny Edge Detection

Aim: To detect object boundaries using Canny edge detection.

Algorithm:

1. Read and convert the image to grayscale.
2. Reduce noise using Gaussian blur.
3. Apply `cv2.Canny()`.
4. Display the edge map.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Strong object boundaries are highlighted.

Experiment 13: Sobel Edge Detection Along X Axis

Aim: To detect edges using the Sobel operator along the X axis.

Algorithm:

1. Read and convert the image to grayscale.
2. Apply `cv2.Sobel()` with `dx=1` and `dy=0`.
3. Convert the result for display.
4. Display the gradient.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Vertical edges are highlighted.

Experiment 14: Sobel Edge Detection Along Y Axis

Aim: To detect edges using the Sobel operator along the Y axis.

Algorithm:

1. Read and convert the image to grayscale.
2. Apply `cv2.Sobel()` with `dx=0` and `dy=1`.
3. Convert the result for display.
4. Display the gradient.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Horizontal edges are highlighted.

Experiment 15: Sobel Edge Detection Along XY Axis

Aim: To detect edges using gradients from both X and Y directions.

Algorithm:

1. Compute the Sobel X gradient.
2. Compute the Sobel Y gradient.
3. Combine both gradients.
4. Display the final result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Edges in both major directions are obtained.

Experiment 16: Laplacian Sharpening with Negative Center Coefficient

Aim: To sharpen an image using a Laplacian mask with a negative center coefficient.

Algorithm:

1. Read the image.
2. Define the negative-center Laplacian kernel.
3. Apply convolution.
4. Combine the response appropriately with the original.
5. Display the sharpened image.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: Edges and fine details are enhanced.

Experiment 17: Laplacian Sharpening with Diagonal Neighbors

Aim: To sharpen an image using an extended Laplacian mask including diagonal neighbors.

Algorithm:

1. Read the image.
2. Define the 8-neighbor Laplacian kernel.
3. Apply `cv2.filter2D()`.
4. Display the sharpened result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Details and edges in all directions are enhanced.

Experiment 18: Laplacian Sharpening with Positive Center Coefficient

Aim: To sharpen an image using a Laplacian mask with a positive center coefficient.

Algorithm:

1. Read the image.
2. Define the positive-center Laplacian mask.
3. Apply convolution.
4. Add the Laplacian response as required.
5. Display the result.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: A sharper image with enhanced local intensity changes is obtained.

Experiment 19: Unsharp Masking

Aim: To sharpen an image using unsharp masking.

Algorithm:

1. Read the original image.
2. Create a blurred version.
3. Subtract blur from the original to obtain a detail mask.
4. Add the mask back to the original.
5. Display the result.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: The image has clearer edges and enhanced details.

Experiment 20: High-Boost Filtering

Aim: To sharpen an image using high-boost filtering.

Algorithm:

1. Read the image.
2. Create a blurred image.
3. Compute the high-frequency mask.
4. Multiply the mask by a boost factor.
5. Add it to the original.
6. Display the result.
7. Import the required Python libraries, mainly OpenCV and NumPy if needed.
8. Read the input image and verify that the image has been loaded successfully.

Output: A strongly sharpened image is obtained.

Experiment 21: Gradient Masking

Aim: To sharpen an image using gradient masks.

Algorithm:

1. Read and convert the image to grayscale.
2. Apply horizontal and vertical gradient masks.
3. Combine gradient responses.
4. Use the result to enhance edges.
5. Display the output.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: Edge clarity and rapid intensity changes are emphasized.

Experiment 22A: Image Watermarking

Aim: To insert a watermark into an image using OpenCV.

Algorithm:

1. Read the main image and watermark text or logo.
2. Choose the watermark position.
3. Blend or overlay the watermark.
4. Save and display the image.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: The original image containing a visible watermark is obtained.

Experiment 22B: Cropping, Copying and Pasting

Aim: To crop a region and copy it into another location or image.

Algorithm:

1. Read the source image.
2. Select the region of interest.
3. Crop the selected region.
4. Copy it.
5. Paste it at the destination position.
6. Display the result.
7. Import the required Python libraries, mainly OpenCV and NumPy if needed.
8. Read the input image and verify that the image has been loaded successfully.

Output: The selected object or region is copied and pasted successfully.

Experiment 23: Boundary Detection Using Convolution Kernel

Aim: To find image boundaries using a convolution-based edge detection kernel.

Algorithm:

1. Read and convert the image to grayscale.
2. Define a boundary detection kernel.
3. Apply `cv2.filter2D()`.
4. Normalize the result.
5. Display the boundary image.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: Object boundaries are highlighted.

Experiment 24: Morphological Erosion

Aim: To perform morphological erosion using OpenCV.

Algorithm:

1. Read a binary or grayscale image.
2. Create a structuring element.
3. Apply `cv2.erode()`.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Foreground regions shrink and small bright noise may be removed.

Experiment 25: Morphological Dilation

Aim: To perform morphological dilation using OpenCV.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply `cv2.dilate()`.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Foreground regions expand and nearby components may connect.

Experiment 26: Morphological Opening

Aim: To perform morphological opening using OpenCV.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply erosion followed by dilation using `cv2.morphologyEx()`.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Small foreground noise is removed while main shapes are preserved.

Experiment 27: Morphological Closing

Aim: To perform morphological closing using OpenCV.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply dilation followed by erosion.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Small holes and gaps inside objects are filled.

Experiment 28: Morphological Gradient

Aim: To perform morphological gradient operation using OpenCV.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply MORPH_GRADIENT.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Object boundaries are highlighted.

Experiment 29: Top-Hat Transformation

Aim: To perform top-hat morphological transformation.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply MORPH_TOPHAT.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Small bright details on a darker background are highlighted.

Experiment 30: Black-Hat Transformation

Aim: To perform black-hat morphological transformation.

Algorithm:

1. Read the image.
2. Create a structuring element.
3. Apply MORPH_BLACKHAT.
4. Display the result.
5. Import the required Python libraries, mainly OpenCV and NumPy if needed.
6. Read the input image and verify that the image has been loaded successfully.
7. Inspect the image size, channels and data type before processing.
8. Convert the image into grayscale or another required format when necessary.

Output: Small dark details on a brighter background are highlighted.

Experiment 31: General Object Recognition – Watch

Aim: To recognize a watch from a given image using OpenCV-based object recognition.

Algorithm:

1. Read the input image.
2. Preprocess by resizing and normalization.
3. Load an object recognition or detection model.
4. Run inference.
5. Obtain the predicted class and confidence.
6. Display the label or bounding box.
7. Import OpenCV and load the required pretrained detection or recognition model.
8. Read the input image or frame and verify that it is loaded correctly.

Output: The watch is recognized and displayed with its predicted label.

Experiment 32: Reverse Video Playback

Aim: To play a video in reverse mode using OpenCV.

Algorithm:

1. Open the video.
2. Read and store frames or obtain total frame count.
3. Access frames from last to first.
4. Display each frame in reverse order.
5. Release resources.
6. Initialize the required video source and check whether it has opened successfully.
7. Read each frame continuously and verify that the frame is valid before processing.
8. Resize or normalize the frame when required for consistent processing.

Output: The input video plays from the last frame to the first.

Experiment 33: Face Detection

Aim: To detect human faces in an image or live video using OpenCV.

Algorithm:

1. Load a pretrained face detector such as Haar Cascade.
2. Read the image or webcam frame.
3. Convert to grayscale if required.
4. Detect face regions.
5. Draw rectangles around faces.
6. Display the result.
7. Import OpenCV and load the required pretrained detection or recognition model.
8. Read the input image or frame and verify that it is loaded correctly.

Output: Detected faces are marked with bounding boxes.

Experiment 34: Vehicle Detection in Video

Aim: To detect vehicles in video frames using OpenCV.

Algorithm:

1. Open the traffic video.
2. Read frames sequentially.
3. Load a pretrained vehicle detector.
4. Detect vehicle regions.
5. Draw bounding boxes.
6. Display the processed video.
7. Initialize the required video source and check whether it has opened successfully.
8. Read each frame continuously and verify that the frame is valid before processing.

Output: Vehicles are highlighted with bounding boxes in the video.

Experiment 35: Draw Rectangle and Extract Objects

Aim: To draw a rectangle around an object and extract the selected object.

Algorithm:

1. Read the image.
2. Choose rectangle coordinates.
3. Draw the rectangle using `cv2.rectangle()`.
4. Crop the object using the same coordinates.
5. Display the marked image and extracted object.
6. Import the required Python libraries, mainly OpenCV and NumPy if needed.
7. Read the input image and verify that the image has been loaded successfully.
8. Inspect the image size, channels and data type before processing.

Output: The selected object is marked and extracted as a separate image.